

Foundations of Generative AI: Six Ways to Model a Distribution

Unit 1 of the Generative AI and LLM crash course

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): intuition and a picture	5
3.2 Tier 2 (medium): the definitions made precise	6
3.2.1 The goal of every generative model	6
3.2.2 Autoregressive models	6
3.2.3 Autoencoders and latent variables	7
3.3 Tier 3 (medium-hard): the main derivations	8
3.3.1 The VAE and its ELBO	8
3.3.2 The GAN and its optimal discriminator	10
3.3.3 Diffusion in one paragraph	10
3.4 Tier 4 (hard): comparing the families, and the four risks	11
3.4.1 Side by side	11
3.4.2 Hallucination, bias, privacy, copyright	11
3.5 Tier 5 (hardest): open questions	12
4. Apply	12
4.1 Worked problems	12
4.2 Code and figures	12
5. Analyze	13

5.1 Limits	13
5.2 Which term dominates	13
5.3 Sensitivity	13
5.4 Failure modes	14
5.5 Checks that the code is right	14
6. Evaluate	14
7. Create	14
8. Visualization lesson	15
9. Problem set	15
10. Retrieval practice and recap	16
11. Where next	16

Session map

Outcome of this session

After this session you can:

1. state the difference between a **discriminative** model $p(y | x)$ and a **generative** model $p(x)$ or $p(x, y)$, and say what each can and cannot do;
2. write the training objective of an autoregressive model, an autoencoder, a VAE, a GAN and a diffusion model in one line each;
3. derive the VAE evidence lower bound (ELBO) and the GAN optimal discriminator, and say what a reparameterisation is for;
4. build and run a character-level text generator, and read its perplexity on training and held-out text;
5. explain hallucination, bias, privacy and copyright risk as consequences of *what generative models are trained to do*, with a measured example.

- **Prerequisites:** Bayes' rule, expectation, the logarithm; gradient descent; the idea of a neural-network layer. Perplexity and next-token prediction are defined in Section 2 below and taken up in depth in Unit 3; the Transformer (Unit 2) is only *named* here.
- **Difficulty ladder used:** easy $\rightarrow$ medium $\rightarrow$ medium-hard $\rightarrow$ hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 5 figures, the module `codes/genmodels/genmodels_np/` (classifiers, a character n -gram model, an MLP with hand-written backpropagation, an autoencoder, a VAE and a GAN; 7 tests including finite-difference gradient checks), and the *Text generator (n -gram)* and *Generative classifier* widgets in the LLM Studio playground.

- **Source anchors:** Kingma and Welling 2013, arXiv:1312.6114 (VAE: Section 2.3 for the bound and Eq. (1), Section 2.4 for the reparameterisation trick); Goodfellow et al. 2014, arXiv:1406.2661 (GAN: Proposition 1 for the optimal discriminator, Theorem 1 for the $-\log 4$ minimum and Eq. (5) for the Jensen-Shannon form); Ho et al. 2020, arXiv:2006.11239 (diffusion: $T = 1000$ steps and variances rising linearly from 10^{-4} to 0.02, Section 4). All three were checked against arXiv and the quoted statements were read in the PDFs.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
discriminative model	$\phi(y x)$	predicts a label from an input	3.1	new
generative model	$p(x)$	models the data itself, so it can create new data	3.1	new
likelihood	$p_\theta(x)$	the probability the model assigns to the observed data	3.2	new
autoregressive model		generates one piece at a time, each conditioned on the earlier pieces	3.2	new
n -gram model		autoregressive model whose context is the last $n - 1$ symbols	3.2	new
autoencoder		encoder squeezes the input to a short code, decoder rebuilds it	3.2	new
latent variable	z	a hidden, lower-dimensional description of the data	3.2	new
variational autoencoder (VAE)		autoencoder whose code is a distribution, trained on the ELBO	3.3	new
ELBO	$\mathcal{L}$	evidence lower bound: reconstruction minus KL	3.3	new
KL divergence	D_{KL}	how different one distribution is from another (not symmetric)	3.3	new
reparameterisation		write $z = \mu + \sigma \epsilon$ so gradients pass through sampling	3.3	new
generative adversarial network (GAN)		generator versus discriminator in a two-player game	3.3	new

Term	Symbol	One-line meaning	Where it appears	New / known
mode collapse		the generator covers only part of the data distribution	3.3	new
diffusion model		learn to remove noise that was added step by step	3.3	new
perplexity	PPL	$e^{\text{cross-entropy}}$: the effective number of equally likely choices	2	new
overfitting / memorisation		reproducing training data instead of its pattern	3.4	new

Terms *used but not taught here*: attention and the Transformer block (Unit 2); tokenization and decoding (Unit 3); the full diffusion derivation (Unit 7).

2. Remember

Discriminative versus generative

A **discriminative** model learns $p(y | x)$: given a photo, is it a cat or a dog? A **generative** model learns the distribution of the data itself, $p(x)$ (or $p(x, y) = p(x | y)p(y)$): what does a cat photo look like, and can I make a new one? A generative classifier can still classify, by Bayes' rule

$$p(y | x) = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}. \quad (1)$$

The six model families in one line each

family	what it learns	training objective
autoregressive	$p(x) = \prod_i p(x_i x_{<i})$	maximise the likelihood (minimise cross-entropy)
autoencoder	a short code z and a decoder	minimise reconstruction error $\ x - \hat{x}\ ^2$
VAE	a distribution over codes $q(z x)$ and a decoder $p(x z)$	maximise the ELBO
GAN	a generator that fools a discriminator	a minimax game
diffusion	to predict and remove noise	regress the added noise at every noise level
Transformer	an <i>architecture</i> (attention blocks), not a family	whatever objective it is trained with

Perplexity (used from Section 3.2.2 on)

Suppose a model gives the true next symbols of a text the probabilities $p_1, \dots, p_n$. Its **cross-entropy** is the average surprise $-\frac{1}{n} \sum_i \ln p_i$ (in nats), and its **perplexity** is $\text{PPL} = e^{\text{cross-entropy}}$: the number of equally likely choices the model is, on average, torn between. Guessing uniformly among V symbols gives $\text{PPL} = V$; a perfect model gives 1; **lower is better**. Worked example: probabilities 0.5, 0.25, 0.125, 0.5 give cross-entropy $\frac{1}{4}(\ln 2 + \ln 4 + \ln 8 + \ln 2) = \frac{1}{4} \ln 128 = 1.213$ and $\text{PPL} = 128^{1/4} = 3.36$.

Memory hooks.

- *Discriminative = judge, generative = forger*. The judge only needs to tell real from fake; the forger must know enough to make something that passes.
- *Autoencoder = a suitcase*. Pack the data into a small suitcase (the code) and unpack it again; whatever does not fit is lost.
- *VAE = a suitcase with a tolerance*. The code is allowed to be fuzzy, which makes the space *between* codes meaningful, so you can sample new ones.
- *GAN = counterfeiter and detective; diffusion = restoring a painting that was gradually buried in noise*.

Recall quiz (answers in Section 10).

1. Which of $p(y | x)$ and $p(x)$ lets you *create* new data?
2. What does an autoregressive model use as its context when it predicts the next symbol?
3. Write the two terms of the VAE loss.
4. What does the optimal GAN discriminator output when the generator is perfect?
5. Name two ways a generative model can leak or misuse its training data.

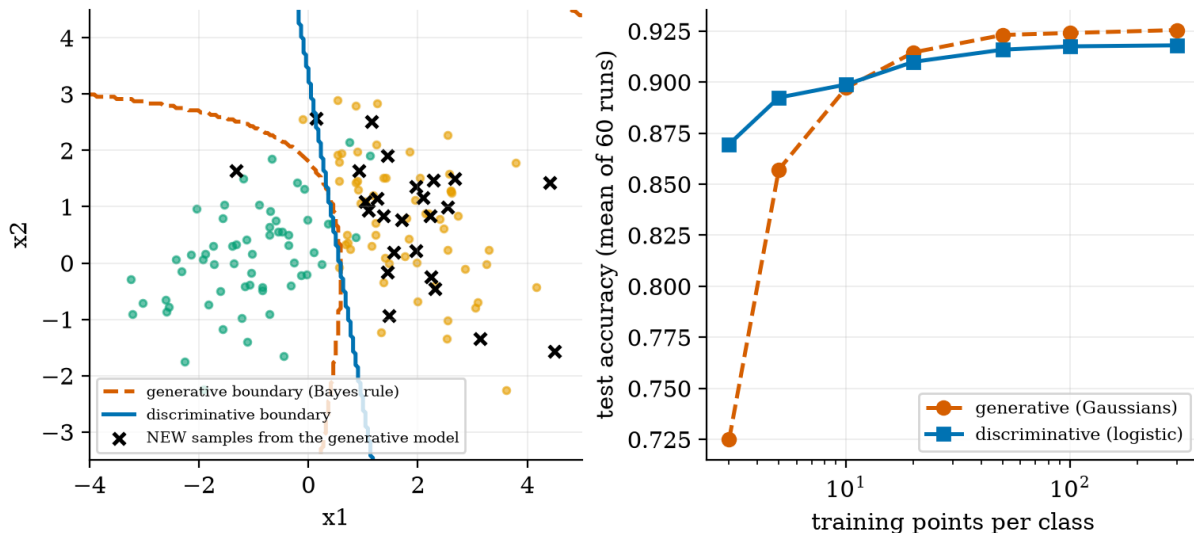
3. Understand: the derivation

3.1 Tier 1 (easy): intuition and a picture

Most machine learning you have met so far is **discriminative**: draw a boundary that separates classes. Generative AI asks for more: *learn what the data looks like*, so that you can produce new examples that are not in the training set.

Figure 1 puts the two side by side on one data set. On the left, a discriminative model (logistic regression, blue) learns only a boundary. A generative model (red dashed boundary; it fits one Gaussian to each class) has *also* learned what each class looks like, so it can **draw new points** (black crosses) from the class it was told to imitate. On the right, the same two models are compared by how fast they learn.

The right-hand panel contradicts a common slogan, so read it carefully. With only 3 training points per class the generative model is *worse* (72.5% against 87.0%): it estimates a full covariance matrix for each class from three points. The two cross at about 10 points per class. With plenty of data the generative model is slightly *better* (92.5% against 91.8% at 300 points per class), because its assumption (each class is Gaussian) happens to be true here, whereas the straight-line boundary of logistic regression cannot follow the curved optimal boundary. Which wins depends on how many parameters the generative model must estimate and on whether its assumptions are right.



Left: two classes of points with the boundary found by a discriminative model (blue, a straight line) and by a generative model that fits one Gaussian per class (red dashed, curved). Black crosses are 25 NEW points drawn from the fitted model of the orange class: only a generative model can do this. Right: test accuracy against the number of training points per class (mean of 60 runs).

3.2 Tier 2 (medium): the definitions made precise

3.2.1 The goal of every generative model

Given samples $x^{(1)}, \dots, x^{(N)}$ from an unknown data distribution p_{data} , build a model p_{θ} that (a) you can sample from, and ideally (b) assigns a **likelihood** $p_{\theta}(x)$ to any x . The standard way to fit it is **maximum likelihood**: choose θ to maximise $\sum_i \ln p_{\theta}(x^{(i)})$. The six families differ in *how they make* p_{θ} tractable.

3.2.2 Autoregressive models

The chain rule of probability lets you write any distribution over a sequence as a product,

$$p_{\theta}(x_{1:n}) = \prod_{i=1}^n p_{\theta}(x_i \mid x_{1:i-1}), \quad (2)$$

the equation on which every language model in Unit 3 is built. Each factor is a distribution over *one* symbol, which is easy to model. Generation is a loop: sample x_1 , then x_2 given x_1 , and so on. Likelihoods are exact. Cost: generation is sequential.

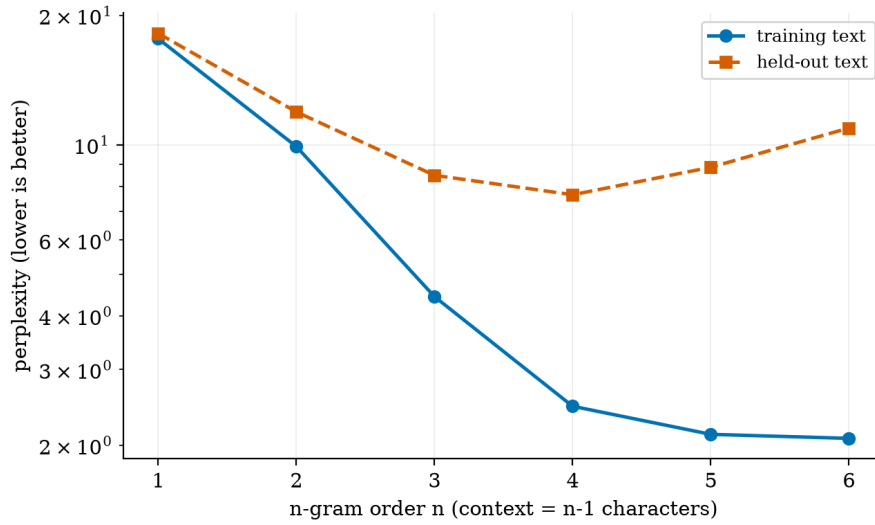
The simplest instance is the **character n -gram model**: the context is only the last $n - 1$ characters, and the probability is a ratio of counts,

$$\hat{p}(c \mid \text{ctx}) = \frac{\#(\text{ctx}, c) + \alpha}{\#(\text{ctx}) + \alpha(V + 1)}, \quad (3)$$

with a small smoothing constant α (here 0.05) so that unseen combinations do not get probability 0, and V the number of distinct characters. This is the “build a simple text-generation model in Python” practical of the syllabus: the module’s `CharNgram` has 40 lines. Trained on the course syllabus (6{,}941 letters and spaces, 27 distinct characters, first 80% for training), it generates:

samples from the same data at four context lengths (seed fixed). Order 1 produces letter soup, order 4 produces *plausible-looking* nonsense (**summariation**), order 6 reads like the syllabus. The last row is not creativity; Section 3.4 shows why.

order n	sample of 110 characters
1	uiv neratltigrwevtanoun iiiuebknmtmdtiniaoraieaauteptlsdeevgiutfer ...
2	sexindietitinntext moun lintecontiltionarodalaivini tis ffulivondichttabunciconiconeenge ...
4	sure former building vs gent evances practical to shot build bpe self attent summariation and moderation modal
6	sure for example text to image systems agent evaluation frameworks deep learning instruction and logging overv



Perplexity of the character n -gram model on the training text (blue) and on held-out text (red dashed), by order n . Training perplexity keeps falling (17.65 at $n = 1$, 2.08 at $n = 6$). Held-out perplexity falls to a minimum of 7.65 at $n = 4$ and then rises again (8.86 at $n = 5$, 10.93 at $n = 6$).

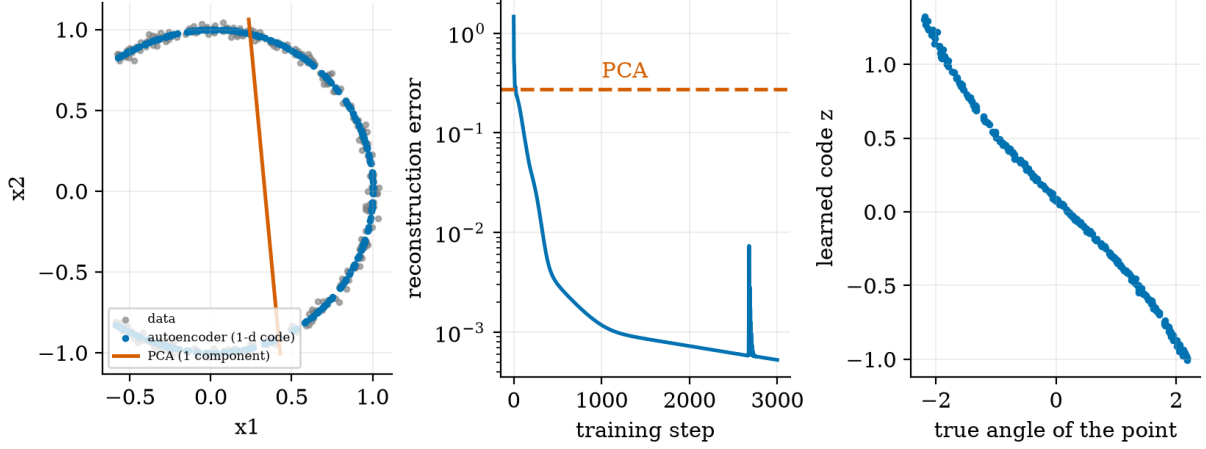
Figure 2 is the first picture of **overfitting**: more context makes the model better on the text it has seen and *worse* on new text. A model that has memorised cannot generalise.

3.2.3 Autoencoders and latent variables

An **autoencoder** has an encoder $e_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^k$ with $k < D$ and a decoder $d_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^D$, trained to minimise

$$\mathcal{L}_{AE} = \frac{1}{N} \sum_i \|x^{(i)} - d_\theta(e_\phi(x^{(i)}))\|^2. \quad (4)$$

The short vector $z = e_\phi(x)$ is a **latent variable**: a compressed, hidden description of the data. The bottleneck forces the network to keep only what matters. A plain autoencoder is a *compressor*, not a generator: nothing says which codes z are valid, so decoding a random z gives garbage. Figure 3 shows what it does learn.



Data on a noisy arc (grey). A nonlinear autoencoder with a one-number code (blue) reconstructs the arc with mean squared error 5.3×10^{-4} ; PCA with one component (red line) can only project onto a straight line and has error 0.275, which is 520 times larger. Right: the learned code z is almost a straight function of the true angle (correlation -0.997).

3.3 Tier 3 (medium-hard): the main derivations

3.3.1 The VAE and its ELBO

A VAE adds the missing ingredient: a **prior** $p(z) = \mathcal{N}(0, I)$ over codes, so that sampling is possible (draw $z \sim p(z)$, decode). The model is $p_\theta(x) = \int p_\theta(x | z) p(z) dz$. This integral is intractable, so introduce an encoder distribution $q_\phi(z | x)$ as a stand-in for the true posterior $p_\theta(z | x)$ and derive a bound that *can* be computed. Start from the definition of $\ln p_\theta(x)$ and insert q_ϕ :

$$\begin{aligned}
 \ln p_\theta(x) &= \mathbb{E}_{q_\phi(z|x)} [\ln p_\theta(x)] && \text{(it does not depend on } z) \\
 &= \mathbb{E}_q \left[\ln \frac{p_\theta(x, z)}{p_\theta(z | x)} \right] && \text{(product rule: } p(x, z) = p(z | x)p(x)) \\
 &= \mathbb{E}_q \left[\ln \frac{p_\theta(x, z)}{q_\phi(z | x)} \right] + \mathbb{E}_q \left[\ln \frac{q_\phi(z | x)}{p_\theta(z | x)} \right] && \text{(multiply and divide by } q) \\
 &= \mathcal{L}(\theta, \phi; x) + D_{KL}(q_\phi(z | x) \| p_\theta(z | x)).
 \end{aligned} \tag{5}$$

This is Eq. (1) of Kingma and Welling. The KL divergence is never negative, so $\ln p_\theta(x) \geq \mathcal{L}$: $\mathcal{L}$ is a **lower bound**, the *evidence lower bound* (ELBO). Expanding $p(x, z) = p(x | z)p(z)$ in the first term gives the form used in practice:

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)} [\ln p_\theta(x | z)]}_{\text{reconstruction}} - \underbrace{D_{KL}(q_\phi(z | x) \| p(z))}_{\text{keep codes close to the prior}}. \tag{6}$$

Two choices make (6) concrete. Take the encoder to be a diagonal Gaussian, $q_\phi(z | x) = \mathcal{N}(\mu, \text{diag } \sigma^2)$, with $\mu, \ln \sigma^2$ output by the encoder network, and the decoder Gaussian with fixed variance σ_x^2 .

The KL term in closed form. For one coordinate, with $p = \mathcal{N}(0, 1)$,

$$\begin{aligned} D_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, 1)) &= \mathbb{E}_q \left[\ln \frac{q(z)}{p(z)} \right] = \mathbb{E}_q \left[-\frac{1}{2} \ln \sigma^2 - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2} \right] \\ &= -\frac{1}{2} \ln \sigma^2 - \frac{1}{2} + \frac{1}{2}(\sigma^2 + \mu^2) \quad (\mathbb{E}[(z-\mu)^2] = \sigma^2, \mathbb{E}[z^2] = \sigma^2 + \mu^2) \\ &= \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \ln \sigma^2). \end{aligned} \quad (7)$$

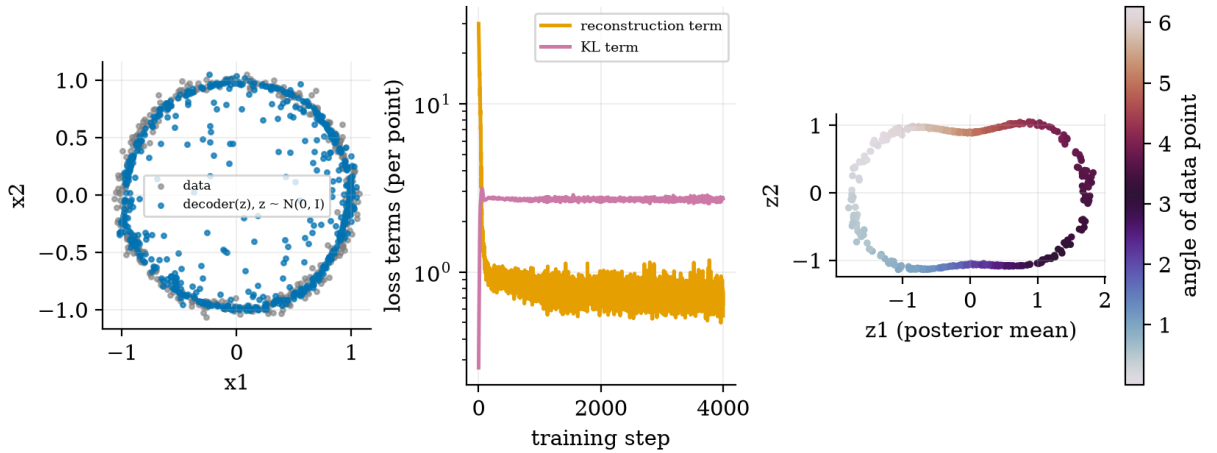
Worked check: $\mu = 1, \sigma = 0.5$ gives $\frac{1}{2}(1 + 0.25 - 1 - \ln 0.25) = \frac{1}{2}(0.25 + 1.3863) = 0.818$. The module’s loss has the value 0 when the encoder outputs $\mu = 0, \ln \sigma^2 = 0$ (test `test_kl_is_zero_for_standard_normal_posterior`).

The reconstruction term. With $p_\theta(x | z) = \mathcal{N}(\hat{x}(z), \sigma_x^2 I)$, $-\ln p_\theta(x | z) = \|x - \hat{x}\|^2 / (2\sigma_x^2) + \text{const.}$ So **minimising $-\mathcal{L}$ is minimising**

$$\frac{\|x - \hat{x}\|^2}{2\sigma_x^2} + \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \ln \sigma_j^2). \quad (8)$$

The first term is the autoencoder loss; the second keeps the codes close to $\mathcal{N}(0, I)$. σ_x^2 sets the trade-off: a small σ_x^2 weights reconstruction heavily.

Reparameterisation. The expectation in (6) is over $z \sim q_\phi$, which depends on ϕ ; a sampled z is a random number, and gradients cannot pass through “sample”. The trick (Kingma and Welling, Section 2.4): write $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$ drawn *independently of the parameters*. Now z is a deterministic, differentiable function of (μ, σ) given ϵ , and ordinary backpropagation works. The module implements it exactly that way, and the test `test_vae_loss_gradients_match_finite_differences` checks the hand-derived gradients of (8) against numerical differentiation.



VAE with a two-dimensional code trained on 400 points of a noisy ring ($\sigma_x^2 = 0.02$, 4000 steps). Left: points decoded from random codes $z \sim \mathcal{N}(0, I)$ (blue) fall on the data ring (grey), with some stray points inside it. Middle: the two loss terms per data point; the KL settles at 2.7 and the reconstruction term at about 0.55. Right: the encoder maps the ring to a closed loop in code space, coloured by angle.

How good are the samples? The data ring has radius 1.001 ± 0.038 ; the VAE samples have radius 0.944 ± 0.119 : on average slightly inside the ring and three times more spread out. That is the familiar **blurriness** of VAEs: the prior fills the code space, including regions the decoder was never trained on.

3.3.2 The GAN and its optimal discriminator

A GAN has a generator $G : z \mapsto x$ with $z \sim p_z$ and a discriminator $D : x \mapsto (0, 1)$, the probability that x is real. They play the game

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\ln D(x)] + \mathbb{E}_{z \sim p_z} [\ln(1 - D(G(z)))]. \quad (9)$$

Optimal discriminator (Proposition 1). Fix G and let p_g be the distribution of $G(z)$. Then $V = \int [p_{\text{data}}(x) \ln D(x) + p_g(x) \ln(1 - D(x))] dx$. Maximise the integrand pointwise: for $a = p_{\text{data}}(x)$ and $b = p_g(x)$, the function $f(D) = a \ln D + b \ln(1 - D)$ has

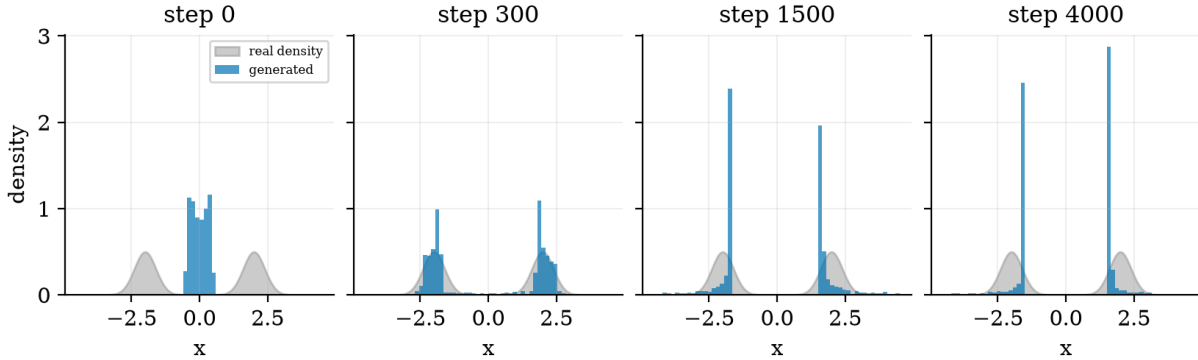
$$f'(D) = \frac{a}{D} - \frac{b}{1 - D} = 0 \implies D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}, \quad (10)$$

and $f'' < 0$, so this is the maximum. Worked check: where $p_{\text{data}} = 0.3$ and $p_g = 0.1$, $D^* = 0.75$: three quarters of the time a point there is real. Substituting back,

$$C(G) = \max_D V(D, G) = -\ln 4 + 2 \text{JSD}(p_{\text{data}} \| p_g) \quad (11)$$

(Eq. (5) of Goodfellow et al.), where JSD is the Jensen-Shannon divergence, which is ≥ 0 and zero only if $p_g = p_{\text{data}}$. So the generator's best possible outcome is $C = -\ln 4 \approx -1.386$ (their Theorem 1), reached when $D^* \equiv \frac{1}{2}$: the discriminator can do no better than guessing.

Practice differs from theory. Early in training D rejects fakes confidently, so $\ln(1 - D(G(z)))$ is flat and gives G almost no gradient. The paper's remedy, used in the module, is the **non-saturating loss**: train G to *maximise* $\ln D(G(z))$ instead. Figure 5 shows a 1-D GAN learning a two-bump target.



A 1-D GAN (two hidden layers of width 24, code size 2) learning a target made of two Gaussians at -2 and $+2$ (grey). At step 0 all generated points are near 0 (standard deviation 0.29). At step 300 both modes are found: left mode mean -1.95 , standard deviation 0.37; right mode 1.98, 0.40; targets are ± 2 and 0.4. At step 1500 and step 4000 both modes are still covered (96% of samples within 1 of a mode at step 4000) but the histograms have become spiky and the modes have drifted to ∓ 1.7 .

Read the picture honestly. The best match to the target is at step 300, not at the end: **GAN training does not converge monotonically**, because two networks chase each other, and no single loss curve tells you when the samples are best. Here the generator keeps both modes; a generator that covers only one is called **mode collapse**, one of the known failure modes.

3.3.3 Diffusion in one paragraph

A diffusion model fixes a *forward* process that adds a little Gaussian noise at each of T steps, $x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_t$, and trains a network to undo it. With $\bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s)$ the noisy sample

can be written directly in terms of the clean one,

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (12)$$

Ho et al. use $T = 1000$ with β_t rising linearly from 10^{-4} to 0.02 (Section 4). Computing the product for that schedule gives $\bar{\alpha}_{100} = 0.897$ (almost no noise yet), $\bar{\alpha}_{500} = 0.079$, and $\bar{\alpha}_{1000} = 4.0 \times 10^{-5}$, so $\sqrt{\bar{\alpha}_{1000}} = 0.0064$: the clean signal is scaled down to under 1% of its original amplitude and what remains is almost pure noise, which is where the reverse process starts. The full derivation, the training loss and a runnable example are in Unit 7.

3.4 Tier 4 (hard): comparing the families, and the four risks

3.4.1 Side by side

a summary of the trade-offs derived above. The statement about diffusion sampling cost is for the original paper’s setting; faster samplers exist and are covered in Unit 7. The Transformer is not a row: it is the neural network inside, most often, an autoregressive model.

	autoregressive	VAE	GAN	diffusion
likelihood	exact	lower bound (ELBO)	none	bound
sampling	one step per symbol, sequential	one forward pass	one forward pass	many steps ($T = 1000$ in the original paper)
training	stable, plain cross-entropy	stable, often blurry	adversarial, can oscillate (Fig. 5)	stable, expensive
typical failure	repetition, hallucination	blur	mode collapse	slow sampling

3.4.2 Hallucination, bias, privacy, copyright

All four risks follow from **what the training objective rewards**.

- **Hallucination.** The objective is the likelihood of the training text, not its truth. A model is rewarded for *plausible* continuations. Section 3.2.2 already shows the effect in miniature: order 4 invents `summariation` and `evances`, which have the statistical look of English words and none of the meaning.
- **Bias.** A model fitted to a distribution reproduces that distribution, including its imbalances and stereotypes. Generative models cannot be more balanced than their data without an extra step.
- **Privacy and copyright through memorisation.** A model with enough capacity can reproduce its training data. In the n -gram model this is measurable: take a 3000-character sample and count the share of its 12-character windows that occur *verbatim* in the training text.

memorisation in the character n -gram model (from `fig02_ngram.py`). It rises together with the *held-out* perplexity in Figure 2: the same capacity that lets the model copy hurts it on new text. Large neural models are different machines, but the mechanism is the same, so training data that contains private or copyrighted text can come back out.

order n	1	2	3	4	6	8
verbatim	0.0%	0.0%	0.1%	6.9%	60.0%	88.8%
12-character windows						

- **Safety.** A model that can generate anything can generate harmful content. Mitigations (filtering data, tuning the model, filtering outputs) are the subject of Unit 9.

3.5 Tier 5 (hardest): open questions

1. **Can a model know when it is hallucinating?** The likelihood a model assigns to a sentence is not a reliable signal of whether it is true. Whether a model’s internal state can tell “I know this” from “I am guessing”, and whether that can be made reliable, is an active research question.
2. **Is good likelihood the same as good samples?** A model can assign high likelihood to data and still produce poor samples, and the reverse (GANs have no likelihood at all yet produced sharp images). How to evaluate generative models without a single trusted number is unsettled.
3. **How much memorisation is acceptable?** There is no agreed threshold at which reproducing training text becomes a privacy or copyright problem, and measures like the one in Section 3.4.2 depend on the window size chosen.

4. Apply

4.1 Worked problems

Problem 1 (Bayes classification). Two classes with priors 0.5 each; at a point x the class-conditional densities are $p(x | A) = 0.2$ and $p(x | B) = 0.05$. By (1), $p(A | x) = \frac{0.2 \cdot 0.5}{0.2 \cdot 0.5 + 0.05 \cdot 0.5} = 0.8$.

Problem 2 (KL). A VAE encoder outputs $\mu = 1$, $\sigma = 0.5$ for one coordinate. By (7) the KL to the prior is 0.818 nats.

Problem 3 (optimal discriminator). At a point where $p_{\text{data}} = 0.3$ and $p_g = 0.1$, $D^* = 0.3/0.4 = 0.75$. If the generator later matches the data, $D^* = 0.5$ everywhere.

Problem 4 (n -gram). In context `th` suppose the training text has `e` 8 times, `a` twice, nothing else. With $\alpha = 0$, $\hat{p}(e | th) = 0.8$ and the loss contribution of a true `e` is $-\ln 0.8 = 0.223$ nats; of a true `i` (never seen) it would be infinite, which is why smoothing is needed.

Problem 5 (diffusion). For $x_0 = 1$ and $\bar{\alpha}_t = 0.5$, (12) gives $x_t = 0.707 + 0.707 \epsilon$: mean 0.707, standard deviation 0.707.

4.2 Code and figures

Everything is in `codes/genmodels/genmodels_np/` (see its README). The generator loop of the n -gram model:

```
def generate(self, length, rng, seed_text=""):
    ctx = ("\x00" * (self.n - 1) + seed_text)[-self.n:]
    for _ in range(length):
        counts = self.counts.get(ctx)
        character = self.distribution.get(ctx, counts)
        chars, cnt = zip(*sorted(counts.items()))
        ch = chars[rng.choice(len(chars), p=np.array(cnt) / sum(cnt))]
        out.append(ch); ctx = (ctx + ch)[-self.n:] # slide the context window
```

and the VAE gradient, derived by hand and verified numerically:

```
dz, gd = dec.backward(cd, r / sigma2 / B) # through the decoder
dmu = dz + mu / B # + d(KL)/d(mu)
dlogv = dz * eps * 0.5 * std + 0.5 * (np.exp(logv) - 1) / B
```

All five figure scripts print the numbers quoted in this document.

Try it

The code `zip` lets you change `n` in `CharNgram`, `sigma2` in `train_vae`, or the real distribution in `train_gan_1d`, and rerun the figure scripts. The playground's *Text generator (n-gram)* widget runs the same model in your browser, and its *Generative classifier* widget lets you move the two Gaussians of Figure 1 yourself. The Unit 3 *Next-token sampler* shows the decoding side of an autoregressive model.

5. Analyze

Checklist items used: limits (5.1), dominance (5.2), sensitivity (5.3), failure modes (5.4), invariants and gradient checks (5.5).

5.1 Limits

- $\alpha \rightarrow \infty$ in (3): every character becomes equally likely (uniform), and the model ignores the data.
- $\sigma_x^2 \rightarrow \infty$ in (8): reconstruction is ignored and the encoder collapses to the prior ($\text{KL} \rightarrow 0$): the code carries no information.
- $\sigma_x^2 \rightarrow 0$: reconstruction dominates and the VAE approaches a plain autoencoder.
- $p_g = p_{\text{data}}$ in the GAN: $D^* \equiv \frac{1}{2}$ and $C = -\ln 4$.

5.2 Which term dominates

In the trained VAE of Figure 4, the KL term (2.7) is about five times the reconstruction term (0.55) per data point, so the prior pull is strong; raising σ_x^2 would raise the share of KL further.

5.3 Sensitivity

Figure 2 is a sensitivity plot in n : the held-out perplexity changes by 43% between $n = 4$ (7.65) and $n = 6$ (10.93). The GAN of Figure 5 is sensitive in *time*: the best step was 300 of 4000.

5.4 Failure modes

- **Memorisation** in autoregressive models (60% verbatim windows at $n = 6$).
- **Blur and stray samples** in VAEs (Figure 4: sample radius spread 0.119 against 0.038 for the data).
- **Instability and drift** in GANs (Figure 5).
- **Compression without structure** in plain autoencoders: decoding a code that was never produced by the encoder gives no guarantee.

5.5 Checks that the code is right

- `test_mlp_backprop_matches_finite_differences` and `test_vae_loss_gradients_match_finite_differences`: hand-written backpropagation agrees with numerical differentiation to relative error 10^{-5} or better.
- `test_kl_is_zero_for_standard_normal_posterior`: the KL is exactly zero at $\mu = 0, \ln \sigma^2 = 0$.
- `test_autoencoder_beats_pca_on_a_curve`, `test_gan_moves_toward_the_real_mean`, `test_ngram_prob_sums_to_one`: the behaviours described above hold.

6. Evaluate

Choosing a family. For text and code the autoregressive Transformer is the standard (Units 2 and 3); for images the leading systems have mostly been diffusion-based (Unit 7); VAEs are useful when you need a smooth, low-dimensional latent space; GANs are the fast-sampling option when you can afford careful training. This is a snapshot, not a law: the ranking has changed several times since 2013 and will change again.

Generative versus discriminative in practice. If you only need a label and have plenty of data, a discriminative model is usually simpler and at least as accurate (Figure 1, right, and the reasoning around it). Choose a generative model when you need samples, a density, or to handle missing data.

When not to trust a generative model. When correctness matters more than plausibility (medical, legal, financial claims), unless the output is checked or grounded in sources (Unit 5); when the training data contain sensitive personal text; when a single number such as perplexity is used to claim that samples are good.

Wider context. The distinction between *learning to discriminate* and *learning to generate* is old in statistics, and so is the debate over which is more fundamental. Generative AI inherits both the power and the unresolved questions: a system that can produce anything is only as trustworthy as the process that checks it.

7. Create

What you can build from this

1. **Your own text generator:** train CharNgram on a book chapter you choose; plot train versus test perplexity by order and pick the order that minimises held-out perplexity.
2. **Memorisation meter:** extend the verbatim-window measure to word windows and to

several window lengths, and plot the curve for orders 2 to 10.

3. **Latent-space walk:** train the VAE on a ring or a spiral, then decode a straight line of codes between two points and plot the path.
4. **GAN stability study:** run `train_gan_1d` with 20 seeds and report how often both modes are found at step 300 versus step 4000.
5. **Combine with Unit 3:** replace the character n -gram by a *word* model using the BPE tokenizer of Unit 3 and compare perplexities.
6. **Module for the open-source package:** add a beta weight on the KL term to `train_vae` and test that $\beta = 0$ gives the autoencoder.

8. Visualization lesson

charts taught here. How to read the small multiples in Figure 5: compare each blue histogram with the grey target in the same panel and look at the *shape*, not just the position: the mean can be right while the spread is wrong.

Chart type	Best for	How it misleads	matplotlib call
scatter with decision contour	classes and a boundary (Fig. 1 left)	the boundary looks certain; it is only the 50% line	<code>ax.contour(X, Y, Z, levels=[.5])</code>
learning curve (accuracy against training size, log x)	data efficiency (Fig. 1 right)	averages hide run-to-run spread	<code>ax.semilogx(n, acc, "o-")</code>
train versus held-out curve	overfitting (Fig. 2)	a held-out set that is too small makes the curve noisy	<code>ax.semilogy(order, ptr); ax.semilogy(order, pte)</code>
small multiples of histograms	a distribution changing over training (Fig. 5)	different bin widths change the apparent shape; here all share bins and the y-axis	<code>plt.subplots(1, 4, sharey=True)</code> and <code>ax.hist(..., range=(-4.5, 4.5))</code>
scatter coloured by a variable	which part of the input maps where (Fig. 4 right)	a cyclic variable such as an angle needs a <i>cyclic</i> colormap	<code>ax.scatter(x, y, c=angle, cmap="twilight")</code>

9. Problem set

Graded exercises

1. (easy) State one task where you need a generative model and one where a discriminative model is enough.
2. (medium) Compute the KL divergence of $\mathcal{N}(2, 1)$ from $\mathcal{N}(0, 1)$ using (7), and of $\mathcal{N}(0, 4)$ from $\mathcal{N}(0, 1)$.
3. (medium-hard) Show that $f(D) = a \ln D + b \ln(1 - D)$ with $a, b > 0$ is maximised at $D = a/(a + b)$, and that the maximum value is $a \ln \frac{a}{a+b} + b \ln \frac{b}{a+b}$.

4. (hard) Derive (12) from the one-step rule $x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_t$ for $t = 2$ and generalise. (Hint: sums of independent Gaussians are Gaussian; add the variances.)
5. (hardest) With $\alpha = 0.05$ and $V = 27$, compute from (3) the total probability the smoothed model assigns to characters *never seen* after a context that occurred $m = 10$ times followed by 3 distinct characters, and again for $m = 1$ followed by 1 character. Explain why this makes smoothing matter more as the order n grows. (Answer: $24 \cdot 0.05 / 11.4 = 0.105$ and $26 \cdot 0.05 / 2.4 = 0.542$.)

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. $p(x)$ (or $p(x | y)$); $p(y | x)$ alone only labels.
2. The previous symbols: all of them in the chain rule, the last $n - 1$ in an n -gram.
3. Reconstruction error $\|x - \hat{x}\|^2 / (2\sigma_x^2)$ and the KL term $\frac{1}{2} \sum (\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$.
4. $\frac{1}{2}$ everywhere: it cannot tell real from generated.
5. Memorisation (reproducing training text, Section 3.4.2) and hallucination or bias reflected from the data; both follow from what the objective rewards.

Explain it to a friend.

- Why can a generative classifier draw new examples but logistic regression cannot?
- Why does adding more context to an n -gram model first help and then hurt on new text?
- What is the trick that lets gradients flow through random sampling in a VAE?

Re-derive tomorrow without looking: (a) $D^* = p_{\text{data}} / (p_{\text{data}} + p_g)$; (b) the VAE KL formula (7); (c) $\ln p(x) = \mathcal{L} + D_{KL}(q \| p(z | x))$.

11. Where next

- **Next:** Unit 2 (neural networks and the Transformer), which opens up the network inside an autoregressive model; Unit 3 then covers tokens, decoding and scaling.
- **Terms met but not taught:** classifier-free guidance, flow matching, energy-based models (named in surveys; not needed for this course).
- **Open threads:** what makes a GAN's best step 300 rather than 4000 (Figure 5; not resolved here); how to measure memorisation in large models (Section 3.5).